

排序思路

- ⚽ 排序主要是针对基于每个用户召回后的topk个候选item进行打分，然后根据分数重新排序，把用户更可能点击的文章排在前面。因此排序模型的输入是当前需要打分的**用户特征**、**候选的item特征**及**两者交互特征**。

输入输出

输入：用户和候选item特征，一个用户如果召回了100个item，则可以组成100个样本输入到排序模型中，基于用户画像和item特征进行打分

输出：用户点击当前候选item的概率

特征工程

各召回通道分数：

代码块

```
1 score_itemcf
2 score_dssm
```

候选item与用户Last 1点击文章的交互特征：

代码块

```
1 user_last_click_timestamp_diff: 点击时间和候选item创建时间之差
2 user_last_click_created_at_ts_diff: 点击item和候选item创建时间差
3 user_last_click_words_count_diff: 字数差
```

候选item属性特征：

代码块

```
1 article_id_cnt: 被点击次数
2 created_at_ts: 创建时间
3 article_id: 文章id
4 words_count: 字数
5 category_id: 类目id
```

用户行为序列统计特征：

代码块

- 1 user_click_last_article_click_time: 最后一次点击时间
- 2 user_click_last_article_created_time: 最后一次点击item的创建时间
- 3 user_click_last_article_words_count: 最后一次点击文章的字数
- 4 user_click_timestamp_created_at_ts_diff_mean: 用户点击item中被点击时间和创建时间间隔的平均值
- 5 user_clicked_article_created_time_max: 点击item中创建时间最大值
- 6 user_click_timestamp_created_at_ts_diff_std: 用户点击item中被点击时间和创建时间间隔的标准差

用户行为习惯特征：

代码块

- 1 user_id_category_id_cnt: 这个用户过去点击历史里，对候选文章所属类别点击过的次数。
- 2 user_id_click_diff_mean: 点击时间间隔平均值
- 3 user_clicked_article_words_count_mean: 点击文章字数平均值
- 4 user_id_click_article_created_at_ts_diff_mean: 点击文章创建时间差（按顺序作差）的平均值
- 5 user_clicked_article_click_time_mean: 点击时间平均值
- 6 user_click_datetime_hour_std: 用户每次点击所在的小时数的标准差
- 7 user_id_cnt: 用户点击次数

用户基础特征：

代码块

- 1 user_id: 用户ID

特征处理

对于id特征，直接转换为embedding向量；对于连续特征，先进行**分桶**，转换成id特征后再进行embedding。这样的处理方式有利于统一特征输入格式（都是ID了），并提升深度模型对特征非线性关系的表达能力。

连续值的缺点：

1. 容易受到**极端值（outliers）**影响

2. 特征在不同数值区间的**分布不均衡**，模型难以学到有效的特征表达

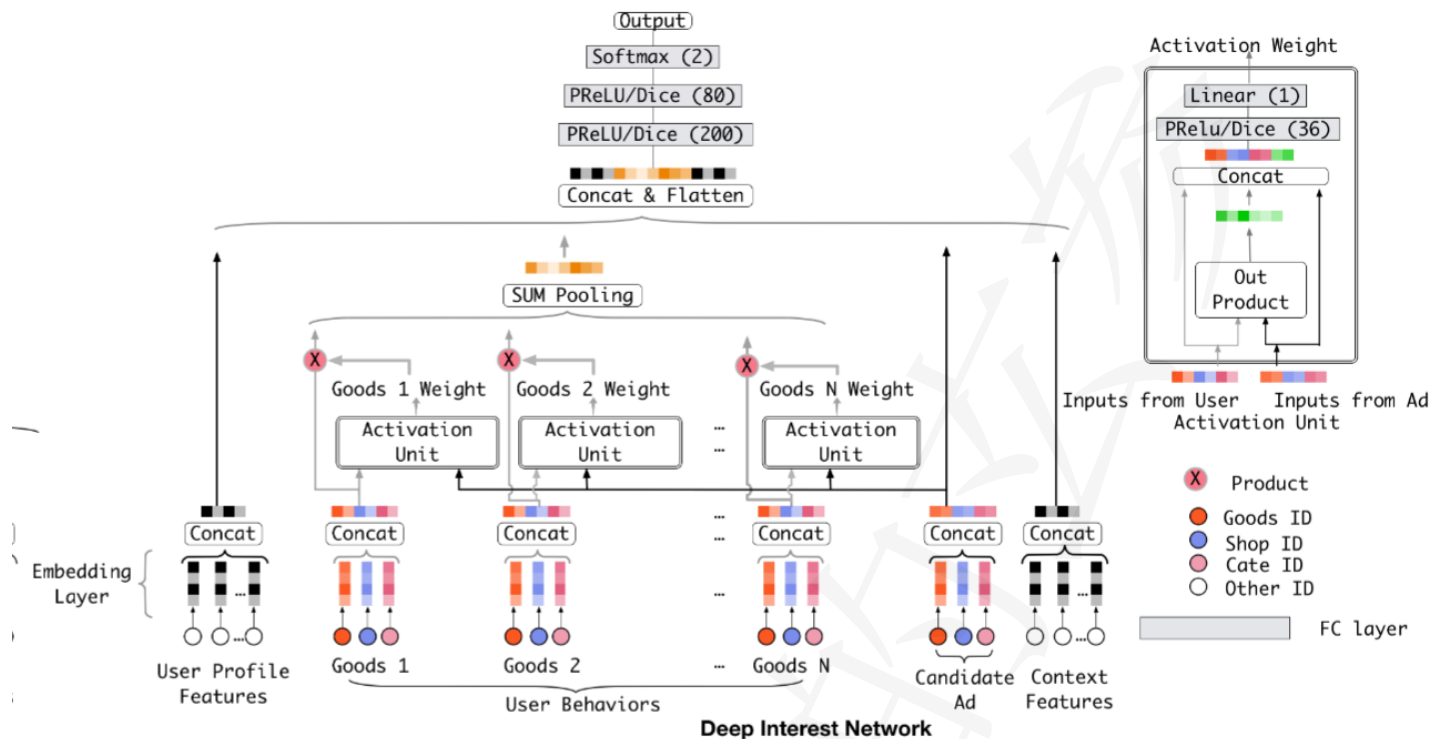
embedding好处：

1. embedding 有更强的表示能力，可以学习**非线性的语义特征**
2. 等频分桶（quantile binning）确保每个桶中样本数量相对均衡，**缓解长尾和数据偏态**
3. 对于 DNN 来说，embedding 层相当于引入了一层非线性变换，提升表达力
4. 等距（equal width）容易出现某些分桶几乎没有样本；
5. 等频分桶可以保证每个桶样本量均衡，训练更稳定，embedding 不容易过拟合；
6. 对于长尾分布（推荐系统中非常常见），等频能更好覆盖主要分布。

DIN

DIN 主要是一个序列建模方法。可以理解成是对交互序列中的ID原始Embedding，融合成一个低维的能代表整个序列特征的语义向量。它利用 **target-aware attention** 机制，根据候选 item 动态激活用户历史行为序列中与其更相关的行为，使得这些行为在聚合过程中的权重更大。这样可以有效捕捉 **用户对当前 item 的局部兴趣偏好**，避免了静态 pooling 的信息损失问题，特别适合序列推荐或 CTR 预估任务。

一些个人理解：比如候选item是特朗普相关的新闻，然后用户的历史点击序列中又存在特朗普或美国相关的新闻，那么针对这个（用户，item）的样本，用户的序列特征就会由这些历史点击中与特朗普相关的新闻加权融合，使得聚合后的embedding带有一定的兴趣偏好，也就是局部兴趣偏好。如果是直接平均pooling的话，对于任何一个候选item，序列特征都是一致的，缺乏对当前候选物品的动态表达能），无法捕捉和激活用户在面对不同物品时所展现出的多样化兴趣，从而导致用户特定的局部兴趣被大量不相关的历史行为所“平滑”或“稀释”掉了。



计算注意力分数

- 将候选新闻的嵌入向量与用户历史交互新闻序列中每个新闻的嵌入向量一同输入激活单元，通过捕捉二者间的语义匹配度，精准计算序列中每条**历史新闻**相对于当前**候选新闻**的**注意力权重**
- 该过程核心是以候选item，动态识别用户历史行为中与当前推荐目标（候选文章）强相关的兴趣点，激活局部的兴趣，适配新闻推荐中用户兴趣易随热点切换的场景。

融合序列embedding

- 依据第一步得到的注意力权重，对用户历史新闻序列的所有嵌入向量进行加权聚合。
- 与候选新闻**相关性越高**的历史新闻，在聚合过程中占据的**权重越大**，最终生成低维稠密的全局序列特征向量。
- 这种动态融合方式避免了传统静态池化（如平均池化）对**用户差异化兴趣**的抹平，能精准提炼用户针对当前候选新闻的**局部兴趣偏好**。

输送下游网络

- 将加权融合后的序列特征向量，与候选新闻自身特征向量、用户静态特征向量进行拼接，共同输入下游 CTR 预估模型（如DNN）完成推荐排序分数预测。

- DCN 的核心思想在于其 Cross 层**不依赖**非线性激活函数，而是通过层级结构的特征外积（或逐元素相乘）与残差连接，**严格且显式**地拟合一定交叉数范围内的输入特征多项式组合。网络每深化一层，特征交叉的阶数便随之递增。
- DCN-V2 是 DCN 的增强版本，通过在**交叉过程中**引入bilinear cross、MoE等结构改进提升了多阶特征交叉建模能力
- 虽然DCN极其擅长做显式的特征交叉，但普通的 DNN 在捕获某些隐式的、高度非线性的泛化模式上依然有优势，因此可结合两者，通过**串行**或**并行**结构组合（如图）

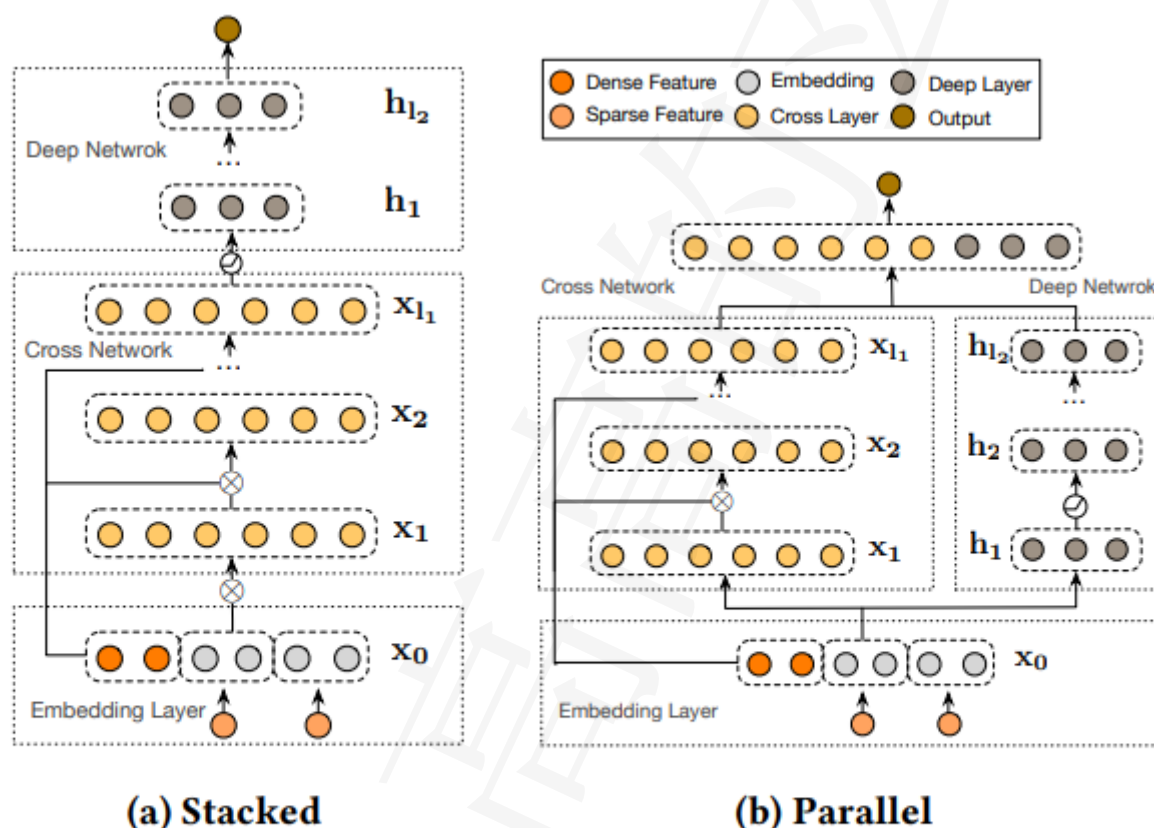


Figure 1: Visualization of DCN-V2. $\otimes$ represents the cross operation in Eq. (1), i.e., $x_{l+1} = x_0 \odot (W_l x_l + b_l) + x_l$.

Cross Network 分支

- 这里设计了一个多阶交叉网络来进行**特征交叉**。
- 正如网络结构图中的连线所示，每一层 l 的输出 x_l 在进入下一层时，都会与初始特征 x_0 进行一次 **bilinear Cross** 交叉操作（即图中的 $\otimes$ ），然后再通过**残差连接**加上该层原本的输入 x_l 得到 x_{l+1} 。

bilinear Cross 操作，就是利用**权重矩阵**显式地对**原始输入特征**和**当前隐藏层特征**进行乘法维度的组合

- 初始时就是 x_0 和 x_0 交叉
- **数学公式表达**：通常可以用公式抽象表示为：

$$x_{l+1} = x_0 \odot (W_l x_l + b_l) + x_l$$

整体结构其实就是一个维度不变的MLP网络，然后每一层之间都有一个 x_0 进行逐项元的相乘交叉和 x_l 的残差连接

权重矩阵低秩分解

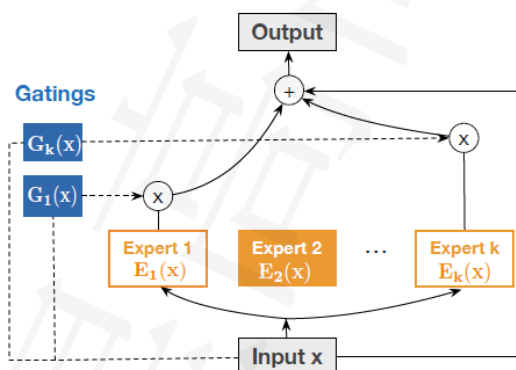
- 将 W_l 分解成两个低秩矩阵， U_l 和 V_l ， $W_l = U_l V_l^T$
- 其中 W_l 维度为 $d \times d$ ， U_l 和 V_l 维度为 $d \times d'$ ，其中 $d' \ll d$

优点：

1. 大幅降低计算成本，满足线上严格的延迟要求
2. 学习的权重矩阵天然具有低秩性，分解后效果损失不大

Mixture of Experts (MoE) 框架

引入**多个专家神经网络**，通过**门控单元**来动态的融合各专家提取的交叉特征，根据样本自动选择更适合的分支，提升模型的适应性和鲁棒性



- 将每一层的交叉网络分成了**多个专家神经网络**，最终残差连接前的输出结果通过对这多个专家神经网络的输出**加权求和**得到。

$$x_{l+1} = \sum_{i=1}^K G_i(x_l) E_i(x_l) + x_l$$

- 基于低秩分解，在低维时加入非线性变换，其中 g 为激活函数， C_l^i 为维度为 d' 的权重矩阵 $E_i(x_l) = x_0 \odot (U_l^i \cdot g(C_l^i \cdot g(V_l^{iT} x_l)) + b_l)$

这么做的目的是充分利用好低维空间，因为本身直接乘是线性的，低秩分解可能会带来一点点损失，而在这里加入一些非线性运算，可以弥补损失，拓宽模型的表达上限

- 门控网络的计算：将输入特征经过简单的全连接层映射到 K 个维度的向量， K 即为专家数量，然后用 softmax 激活函数得到 K 个加和为 1 的 gate 权重，也可以对每个元素进行 sigmoid，得到加和不为 1 的 gate 权重

优点

- 能**显式地学习高阶特征组合形式**，拟合效率远超传统 DNN 的隐式学习。
- 使用权重矩阵替代一维向量（相较于 V1 版本），大幅**提升了模型对复杂特征交互**的建模能力。
- 通过引入低秩矩阵分解和混合专家机制（MoE），在**维持甚至超越基线精度**的前提下，大幅**降低了参数量、计算量（FLOPs）和线上推理延迟**。
- 核心计算仅为基础的矩阵乘法与逐元素相乘，无复杂的定制化算子，较易通过代码实现。